Jobsheet: Spatial Analysis of NASA FIRMS VIIRS Fire Hotspots in Indonesia (Aug–Sep 2026)

@syntheralab
Topic: Geospatial data analysis with Python (GeoPandas, Matplotlib, scikit-learn, PyDeck)

1. Learning Objectives

1. Load and inspect a NASA FIRMS VIIRS active-fire CSV file with pandas.
2. Clean satellite detection data by removing duplicates and low-confidence points.
3. Convert tabular latitude/longitude data into a GeoDataFrame with a coordinate reference system (CRS).
4. Perform a spatial join to assign each hotspot to an Indonesian province.
5. Normalize hotspot counts by province area using an equal-area projection.
6. Visualize results with bar charts, choropleth maps, time series, and an interactive 3D hexagon map.
7. Detect hotspot clusters with DBSCAN and compare fire intensity using Fire Radiative Power (FRP).
8. Interpret the results and state the limitations of satellite hotspot data.

2. Background

2.1 Why fire hotspots matter

Every year, the dry season in Indonesia brings forest, peatland, and agricultural fires, especially in Kalimantan and Sumatra. These fires affect air quality, health, biodiversity, and greenhouse gas emissions. Satellites let analysts monitor fires over large areas almost in real time.

2.2 NASA FIRMS and VIIRS

NASA's Fire Information for Resource Management System (FIRMS) distributes active-fire detections from satellite sensors. The VIIRS (Visible Infrared Imaging Radiometer Suite) instrument detects thermal anomalies at roughly 375 m resolution. Each detection (a *hotspot*) is a point with a location, acquisition date and time, brightness temperature, confidence level, and Fire Radiative Power (FRP, in MW).

Important columns used in this jobsheet:

Column	Meaning
`latitude`, `longitude`	Location of the detection
`acq_date`	Date of acquisition
`confidence`	Detection confidence: `l` (low), `n` (nominal), `h` (high)
`frp`	Fire Radiative Power in megawatts, a proxy for fire intensity

2.3 Key spatial concepts

- **CRS (Coordinate Reference System):** defines how coordinates map to the Earth. EPSG:4326 is geographic longitude/latitude (degrees).
- **Spatial join:** attaches attributes from one layer to another based on spatial relationships, here "point within polygon".
- **Normalization:** raw counts favor large provinces. Dividing by area gives density (hotspots per 1,000 km²). Area must be computed in an equal-area CRS (EPSG:6933), not in degrees.
- **Choropleth map:** a map where regions are colored by a value.
- **DBSCAN:** a density-based clustering algorithm that groups nearby points and labels isolated points as noise (`-1`).

3. Tools and Materials

Software

- Python 3.10 or newer, Jupyter Notebook or JupyterLab
- Libraries: `pandas`, `geopandas`, `matplotlib`, `numpy`, `scikit-learn`, `folium`, `mapclassify`, `pydeck`

Data

1. NASA FIRMS VIIRS active-fire CSV for Indonesia, 1 Aug – 30 Sep 2026 (file used in the notebook: `fire_nrt_SV-C2_818013.csv`).
2. GADM v4.1 province boundaries for Indonesia, level 1 (`gadm41_IDN_1.shp` with its companion `.shx`, `.dbf`, and `.prj` files).

Install the libraries if needed:

```
pip install pandas geopandas matplotlib numpy scikit-learn folium mapclassify pydeck
```

4. Safety and Good Practice Notes

- Keep all shapefile components (`.shp`, `.shx`, `.dbf`, `.prj`) in the same folder.
- Use raw strings (`r"C:\path\file.csv"`) for Windows paths, or forward slashes.
- Do not embed very large interactive maps in the notebook. Sample the points or save the map to an HTML file.

5. Procedure

Step 1. Load the hotspot data

```
import pandas as pd
import geopandas as gpd

df = pd.read_csv(r"C:\Users\USER\OneDrive\Desktop\NASA\fire_nrt_SV-C2_818013.csv")
print(df.shape)
df.head()
```

Task: Report the number of rows and columns. List the columns you will use later.

Step 2. Inspect and clean the data

```
df.info()
print(df.isna().sum())

df["acq_date"] = pd.to_datetime(df["acq_date"])
print(df["acq_date"].min(), "to", df["acq_date"].max())
print(df["confidence"].value_counts())

df = df.drop_duplicates()
df = df[df["confidence"] != "l"]   # remove low-confidence detections
```

Explanation: Duplicates inflate counts, and low-confidence points are more likely to be false alarms (for example, sun glint or hot industrial surfaces). The date check confirms that the data covers 1 Aug – 30 Sep 2026.

Task: How many rows were removed in total? What percentage of the original data does that represent?

Step 3. Create a GeoDataFrame

```
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df.longitude, df.latitude),
    crs="EPSG:4326",
)
```

Explanation: Each row becomes a point geometry. Always set the CRS explicitly. VIIRS coordinates are in WGS84 (EPSG:4326).

Step 4. Explore the time pattern

```
import matplotlib.pyplot as plt

daily = df.groupby("acq_date").size()
daily.plot(figsize=(10, 4), title="Hotspot count per day")
plt.show()
```

Task: On which dates does the count peak? Does the trend rise toward the end of September?

Step 5. Quick maps of the points

```
%pip install folium mapclassify

# Interactive map of a sample (keeps the notebook light)
gdf.sample(min(5000, len(gdf))).explore(column="frp", cmap="YlOrRd", legend=True)

# Static map of all points
gdf.plot(markersize=0.5, figsize=(10, 6))
```

Explanation: Sampling to 5,000 points avoids a slow, heavy interactive map. The static plot shows the full spatial pattern.

Step 6. Load province boundaries

```
import os

folder = r"C:\Users\USER\OneDrive\Desktop\NASA"
```

```
print([f for f in os.listdir(folder) if "gadm" in f.lower()])

prov = gpd.read_file(os.path.join(folder, "gadm41_IDN_1.shp"))
print(prov.shape)           # about 38 provinces expected
print(prov.crs)             # should be EPSG:4326
print(prov["NAME_1"].head())
```

Task: Confirm the number of provinces and the CRS of the boundary layer.

Step 7. Spatial join: assign hotspots to provinces

```
# Make sure both layers use the same CRS
prov = prov.to_crs(gdf.crs)

joined = gpd.sjoin(gdf, prov[["NAME_1", "geometry"]], predicate="within",
how="inner")
print(len(gdf), "hotspots ->", len(joined), "matched to a province")
```

Explanation: `predicate="within"` keeps points that fall inside a province polygon. Points over the sea or outside the boundary file are dropped by `how="inner"`.

Task: How many hotspots were not matched to any province? Suggest why.

Step 8. Count hotspots per province

```
per_prov = joined.groupby("NAME_1").size().sort_values(ascending=False)
print(per_prov.head(10))

# Share of total (%)
print((per_prov / per_prov.sum() * 100).round(1).head(10))
```

```
# Bar chart of the top 10 provinces
per_prov.head(10).sort_values().plot(
    kind="barh", figsize=(8, 5), title="Top 10 provinces by hotspot count"
)
plt.xlabel("Hotspot count")
plt.show()
```

Step 9. Choropleth of raw counts

```
prov["count"] = prov["NAME_1"].map(per_prov).fillna(0)

fig, ax = plt.subplots(figsize=(12, 6))
prov.plot(column="count", cmap="OrRd", legend=True,
          edgecolor="grey", linewidth=0.3, ax=ax)
ax.set_title("Hotspot count per province")
ax.set_axis_off()
plt.show()
```

Step 10. Normalize by province area

```
# Project to an equal-area CRS before measuring area
prov_area = prov.to_crs("EPSG:6933")
prov["area_km2"] = prov_area.geometry.area / 1e6

prov["per_1000km2"] = prov["count"] / prov["area_km2"] * 1000
```

```
print(prov.sort_values("per_1000km2", ascending=False)
          [["NAME_1", "count", "per_1000km2"]].head(10))
```

```
fig, ax = plt.subplots(figsize=(12, 6))
prov.plot(column="per_1000km2", cmap="OrRd", legend=True,
          edgecolor="grey", linewidth=0.3, ax=ax)
ax.set_title("Hotspot density per 1,000 km² (Aug–Sep 2026)")
ax.set_axis_off()
plt.show()
```

Explanation: Area must never be computed in degrees. EPSG:6933 is an equal-area projection, so the area values in km² are reliable.

Task: Compare the ranking from Step 8 (raw counts) with the ranking here (density). Which provinces move up or down, and why?

Step 11. Weekly trend of the top provinces

```
top4 = per_prov.head(4).index
weekly = (joined[joined["NAME_1"].isin(top4)]
          .groupby([pd.Grouper(key="acq_date", freq="W"), "NAME_1"])
          .size().unstack())
weekly.plot(figsize=(10, 4), title="Weekly hotspots, top provinces")
plt.show()
```

Task: Describe how the fire season builds up in each of the four provinces.

Step 12. Fire intensity (FRP) per province

```
frp_prov = (joined.groupby("NAME_1")["frp"]
            .agg(["count", "mean", "median", "max"])
            .sort_values("count", ascending=False)
            .head(10))
print(frp_prov.round(1))
```

Explanation: The count tells how many fires were detected, while FRP indicates how intense they were. A province can have many weak detections or few strong ones.

Step 13. Cluster hotspots with DBSCAN (Kalimantan subset)

```
from sklearn.cluster import DBSCAN
import numpy as np

# Rough bounding box for Kalimantan
sub = df[(df.longitude.between(108, 119)) & (df.latitude.between(-4.5, 2))]

coords = np.radians(sub[["latitude", "longitude"]].values)
db = DBSCAN(eps=5/6371, min_samples=30, metric="haversine").fit(coords)

sub = sub.assign(cluster=db.labels_)
print(sub["cluster"].value_counts().head(10))   # -1 = noise
```

Explanation: With the haversine metric, `eps` is in radians. Dividing 5 km by the Earth's radius (6,371 km) gives a 5 km neighborhood. A cluster needs at least 30 points. Points labeled `-1` are noise.

Task: How many clusters were found? Try `eps = 3 km` and `10 km` and describe the effect.

Step 14. 3D hexagon map with PyDeck

```
%pip install pydeck

import pydeck as pdk

pts = df[["longitude", "latitude"]].copy()

layer = pdk.Layer(
    "HexagonLayer",
    data=pts,
    get_position="[longitude, latitude]",
    radius=15000,          # hexagon radius in meters
    elevation_scale=10,    # increase for taller bars
    extruded=True,         # makes the hexagons 3D
    coverage=0.9,
    pickable=True,
    auto_highlight=True,
)

view = pdk.ViewState(longitude=118, latitude=-2, zoom=4, pitch=50, bearing=0)

deck = pdk.Deck(
    layers=[layer],
    initial_view_state=view,
    tooltip={"text": "Hotspots in this cell: {elevationValue}"},
)

# Save to a file only; do not embed the map inside the notebook
deck.to_html("hotspots_3d.html", open_browser=False, notebook_display=False)
```

```
import os
print(os.getcwd())   # folder where hotspots_3d.html is saved
```

Explanation: Each hexagon covers a 15 km radius and its height shows the number of hotspots inside. Open `hotspots_3d.html` in a browser and drag to rotate the view.

6. Expected Results

Using the Aug–Sep 2026 data, the notebook produced the following main findings:

Rank	Province	Density (hotspots per 1,000 km²)
1	Central Kalimantan	≈ 654 (about 100,452 detections in total)
2	South Kalimantan	≈ 439
3	West Kalimantan	≈ 426
4	South Sumatra	≈ 320

- Kalimantan provinces dominate the top ranks of the normalized density.
- Papua has a large raw count (about 40,000) but drops out of the top 10 once normalized by area. Raw counts alone are therefore misleading.

- The 3D hexagon map shows the tallest columns over southern and central Kalimantan and southern Sumatra.

Your exact numbers may differ slightly depending on the cleaning rules and the version of the boundary data.

7. Analysis and Discussion

1. **Why normalize?** Large provinces such as Papua accumulate many detections simply because they are big. Density per 1,000 km² makes provinces comparable.
2. **Why Kalimantan and South Sumatra?** Extensive peatland and plantation landscapes, land clearing by burning, and the dry season (often intensified by El Niño conditions) make these areas fire-prone.
3. **Count vs. intensity.** Comparing counts with FRP statistics shows whether a province has many small fires or fewer, more intense ones.
4. **Clusters.** DBSCAN highlights dense fire complexes that may need priority for ground response.

8. Limitations

- The data covers only two months of the peak dry season, so the results should not be generalized to the whole year.
- Hotspots are satellite detections, not burned area. One large fire can produce many points.
- Some detections, especially in eastern Indonesia, may be savanna or agricultural burning rather than forest fires.
- Removing low-confidence points reduces false alarms but may also drop some real small fires.
- Province boundaries are coarse (GADM level 1). Finer units (regency or peat hydrological units) would give more precise results.

9. Exercises

1. Repeat the analysis for the **high-confidence only** subset (`confidence == "h"`). How does the top-10 ranking change?
2. Create a choropleth of **median FRP** per province and compare it with the density map.
3. Add the **day/night** column (`daynight`) and compare hotspot counts between day and night.
4. Run DBSCAN on **Sumatra** (approximately longitude 95–106, latitude −6 to 6) and report the five largest clusters.
5. Change the PyDeck `radius` to 8000 and 30000. Explain how the visual interpretation changes.

10. Report Requirements

Submit a report (PDF or Word) that includes:

1. Title, name, class, and date.
2. Objectives and a short description of the dataset.
3. Screenshots of the main outputs: daily trend, top-10 bar chart, both choropleth maps, weekly trend, and the 3D map.
4. Answers to every **Task** question in Section 5.

5. Analysis, limitations, and conclusion in your own words.
6. The final notebook (`.ipynb`) and `hotspots_3d.html`.

11. Assessment Rubric

Component	Weight
Data loading and cleaning	15%
Spatial join and aggregation	20%
Normalization and maps	20%
Advanced analysis (FRP, DBSCAN, 3D map)	20%
Interpretation and limitations	15%
Report quality and neatness	10%

12. Conclusion

This jobsheet walked through a complete geospatial workflow: from raw NASA FIRMS VIIRS detections to cleaned points, a spatial join with province boundaries, area-normalized density, and advanced views using FRP, DBSCAN clustering, and a 3D map. The main lesson is that normalization changes the story: Central Kalimantan, South Kalimantan, West Kalimantan, and South Sumatra stand out by density, while Papua, which has a high raw count, does not. Always interpret hotspot data together with its limitations.

13. References

1. NASA FIRMS – Fire Information for Resource Management System. https://firms.modaps.eosdis.nasa.gov
2. GADM – Database of Global Administrative Areas, version 4.1. https://gadm.org
3. GeoPandas documentation. https://geopandas.org
4. scikit-learn documentation: DBSCAN. https://scikit-learn.org
5. PyDeck documentation. https://deckgl.readthedocs.io